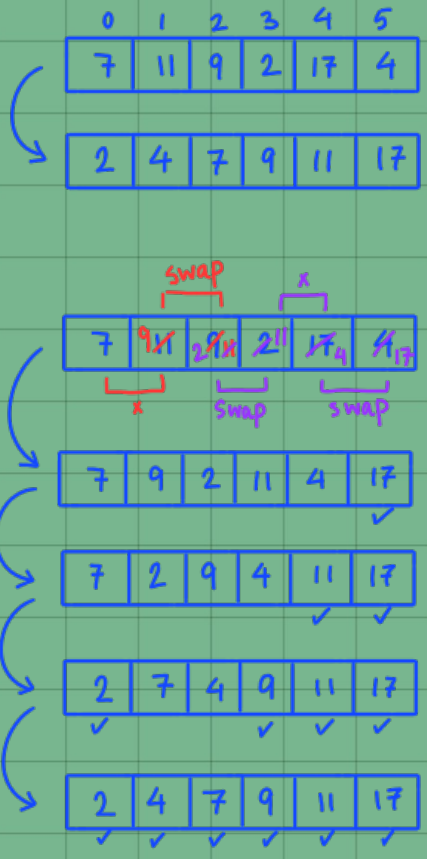
* Sorting Algorithm :-

- Ascending Order: 5,1,4,2,3 → 1,2,3,4,5
- Descending order: 5,1,4,2,3 → 5,4,3,2,1

• Analysis criteria :-

① Time Complexity
② Space Complexity (Inplace Sorting Algorithm)
↳ No extra storage used
③ Stability: 6 1 2 6 → Stable sorting algo → 1 2 6 6
④ Internal sorting algo → all data is loaded into the memory
External SA → all data is not loaded into the memory.
⑤ Adaptive → already sorted data takes less time
⑥ Recursive / Non-recursive SA
↳ use recursion

* Bubble Sort :-



length of array = 6 (n)

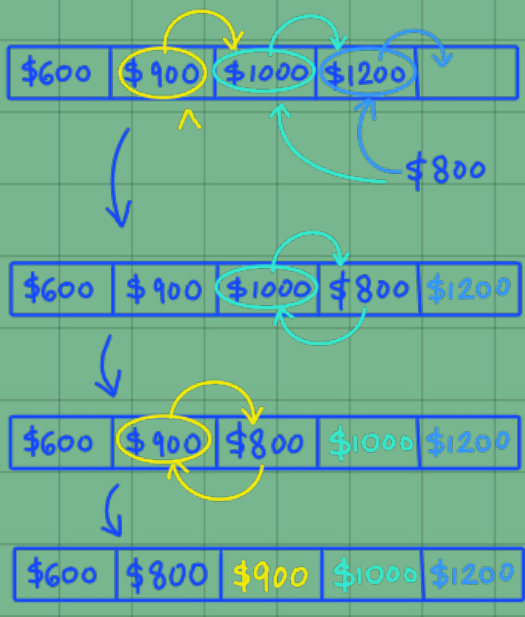
max^m no of passes = (n-1)

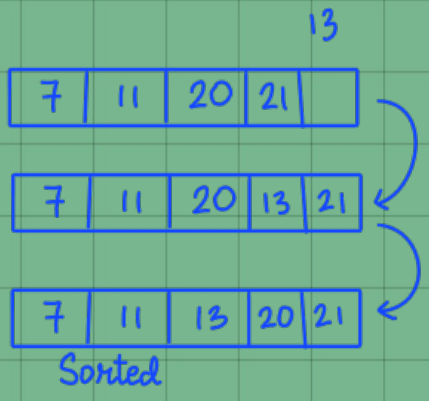
(n-i) comparisons → For a single pass (i^th pass)
↳ (n-i) possible swaps

↓
bcz with every single pass atleast one element get sorted

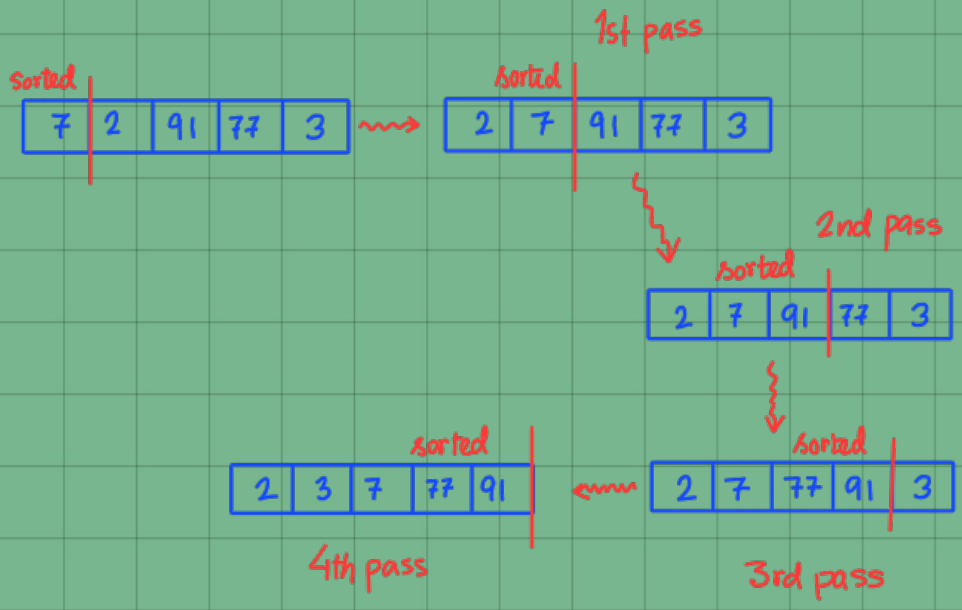
- Time Complexity: $O(n^2)$ $O(n)$
- Stable Algorithm
↳ for already sorted array

* Insertion Sort :-





inserting an element in a sorted array

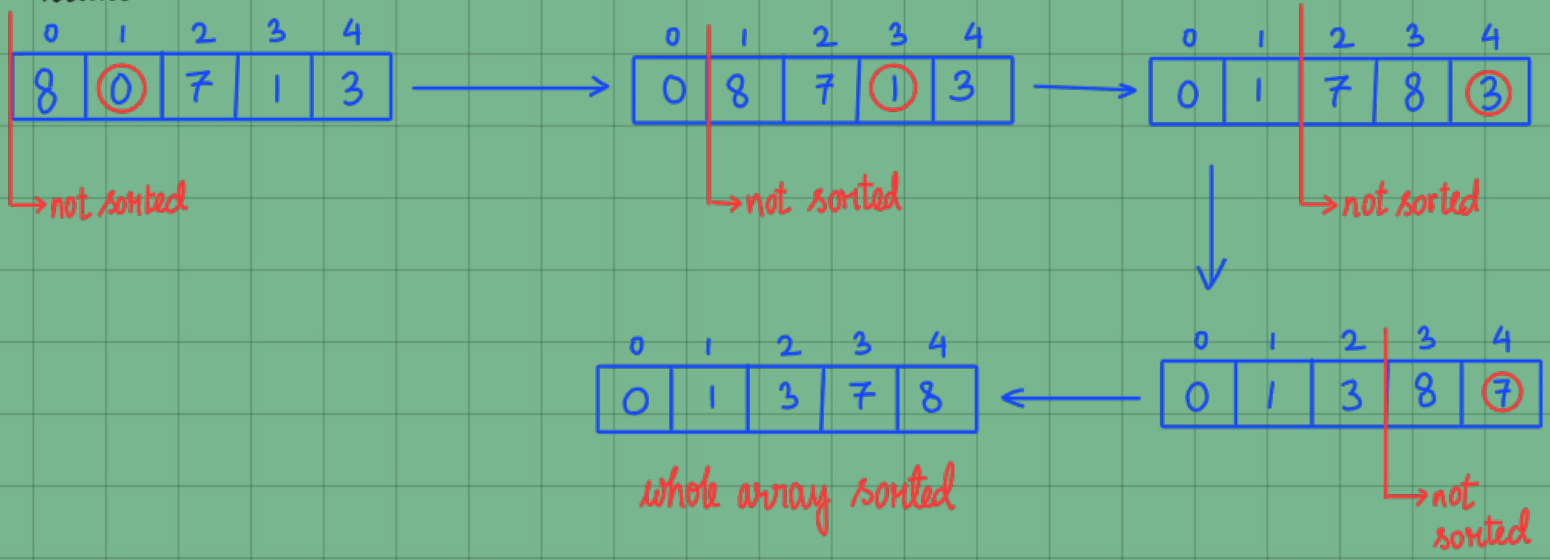


- length of array = n
- no of passes = (n-1)

i^{th} possible comparison
i^{th} possible swaps
↳ in each ith pass
$(i = 0; i < (n-1); i++)$

- Time Complexity : $O(n^2)$ & $O(n)$
  - $O(n^2)$ ↓ avg & worst case
  - $O(n)$ Best Case
- Stable sort
- Adaptive

* Selection Sort :-

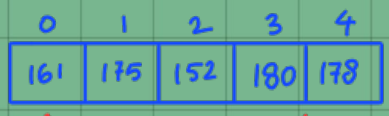


- length of array = n
- no. of passes = (n-1)

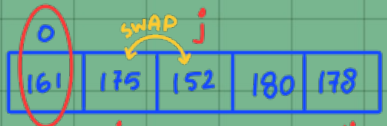
idx=0 1st pass = 4 comparisons (max^m)
idx=1 2nd pass = 3 "
idx=2 3rd pass = 2 "
idx=3,4 4th pass = 1 "

- passes = i to (n-1)
- Time complexity : $O(n^2)$
- Max^m no. of swaps = (n-1)
- "Not" a stable sort
- "Not" an adaptive sort
- sorting in min. no. of swaps

* Quick Sort :-

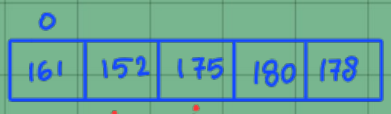


↑ lesser than this on its left side
bigger than this on its right side



② to check higher element

① to check lower element



j < i → stop j

check higher element through traversal of i



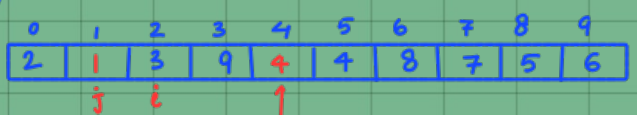
Partially sort (Partition)



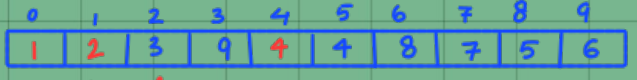
to find higher elements

i & j interchange

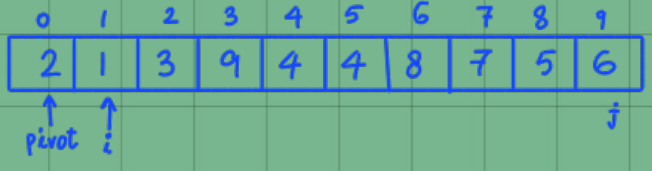
to find lower elements

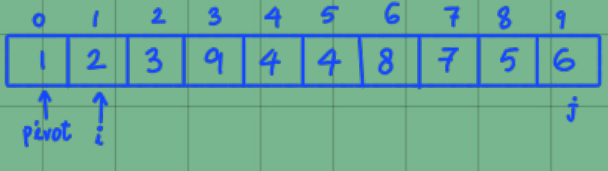


* pivot interchange with j



• Partitioning :-





1> i = low , j = high
2> pivot = low - 1
3> i++ until element > pivot is found
4> j-- until element ≤ pivot is found
5> swap arr[i] & arr[j] and repeat 3&4 until j ≤ i
6> swap pivot & arr[j]

{1} {3,9,4,4,8,7,5,6}

pivot

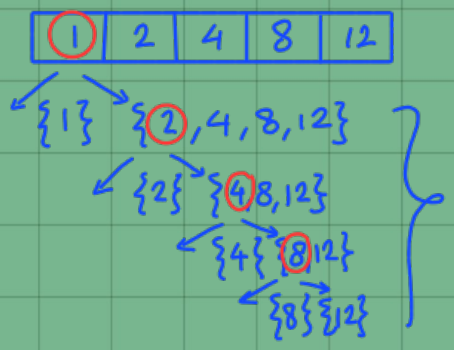
{3} {9,4,4,8,7,5,6}

{6,4,4,8,7,5,9}

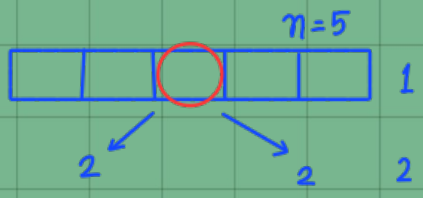
{6,4,4,8,7,5} {9}

Divide & Conquer

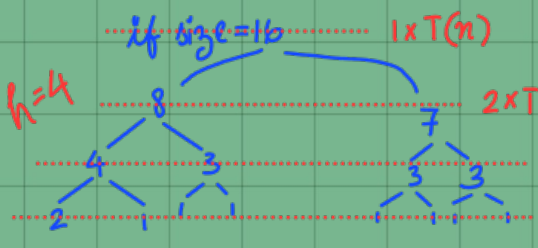
* Analysis Of Quick Sort :-



- Worst Case (↳ already sorted array) $\} O\{(n-1)(k_1n+k_2)\} \approx O(n^2)$
- Time Complexity of partition algorithm
  ↳ some linear fn on of n
  $= k_1 n + k_2$



- Best Case $\approx O(n \log n)$



- Partition Time : $k_1 n + k_2$
  $T(n)$
- Total time : $(k_1 n) + 2(k_1 \frac{n}{2}) + 2^2(k_1 \frac{n}{2^2})$ + ... + h times
  ↓ height of the tree
  $= h \times k_1 n + k \approx O(n \log n)$

$h = 4 = \log_2 16$

- Average Case : O(nlogn)
- Quick sort is not stable (is an inplace algorithm)

* Count Sort : (one of the fastest)

0	1	2	3	4	5	6
3	1	9	7	1	2	4

max element = 9 (given array size = 7 (n))

i →

0	1	2	3	4	5	6	7	8	9
0	~~1~~ 2	~~1~~ 0	~~1~~ 0	~~1~~ 0	0	0	~~1~~ 0	0	~~1~~ 0

count = 10 (m)

j →

- Traverse the main array & element = cnt array[element] → increment

0	1	2	3	4	5	6
1	1	2	3	4	7	9

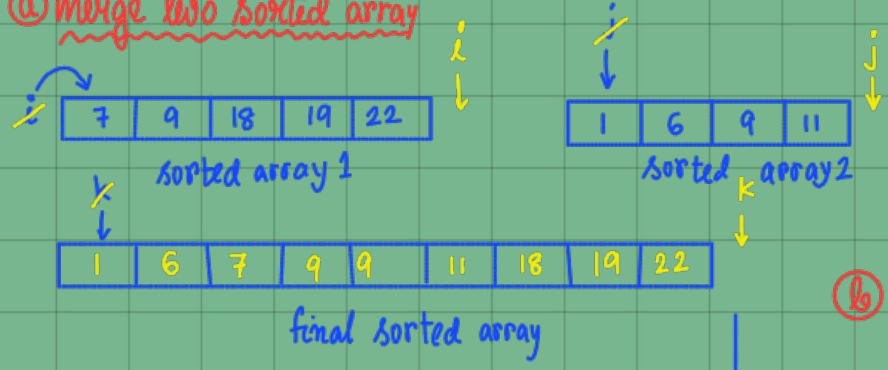
↳ sorted array

put index of count array in this new array until the element at that index is 0

- Time Complexity : $O(n+m)$ (given + cnt array) ↗ linear time complexity at cost of extra space.

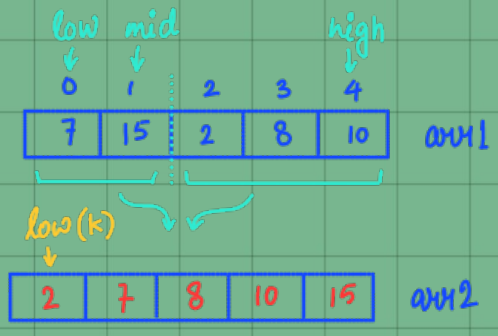
* Merge Sort :-

ⓐ merge two sorted array




```
void merge (A[], B[], m, n){
    int i, j, k;
    i = j = k = 0;
    while (i<m && j<n){
        if (A[i] < B[j]){
            C[k] = A[i];
            i++, k++;
        }
        else{
            C[k] = B[j];
            j++, k++;
        }
    }
    while(i<m){                    } copy all remaining
        C[k] = A[i];               } elements from A to C
        k++, i++;
    }
    while (j<n){                   }
        C[k] = B[j];               } from B to C
        k++, j++;
    }
```

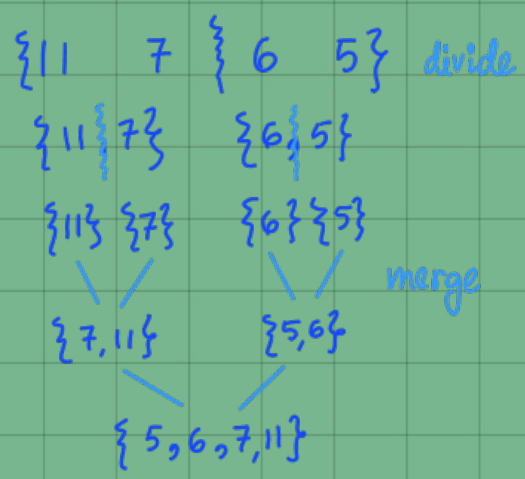
ⓑ merging in a single array




```
void merge (A[], low, mid, high){
    int i, j, k;
    i = low, j = mid+1, k = low;
    while (i<mid && j<high){
        if (A[i] < A[j]){
            B[k] = A[i];
            i++, k++;
        }
        else{
            B[k] = A[j];
            j++, k++;
        }
    }
    while (i<=mid){
        B[k] = A[i];
        i++, k++;
    }
    while (j<=high){
        B[k] = A[j];
        j++, k++;
    }
```

• copy sorted elements from arr2 to arr1

```
for(i=low; i<=high; i++){
    A[i] = B[i];
}
```




```
void merge_sort(int A[], int low, int high){
    if(low< high){
        mid = (low+high)/2;
        merge_sort (A, l, mid);
        merge_sort (A, mid+1, high);
        merge (A,l,mid,high);
    }
}
```